CS 2006 Operating Systems Spring 2026
Assignment 01
BS - CS

Instructions:

- **Individual Assignment:** This is an individual task. Collaboration or sharing of code logic is strictly prohibited.
- **Plagiarism Policy:** All parties involved in any form of plagiarism or cheating will be awarded zero marks.
- **Deadline:** Late submissions are not allowed, and the deadline will not be extended under any circumstances.
- **Submission Format:** Create a new directory named in the following format: BCS Section_RollNumberName_Fullname (e.g. BCS Section_24i-xxxx_FullName).
- Place all source files (.sh, .cpp, .h), and the Makefile into respective question sub directory (e.g. rollno_Q1) Not following the naming convention will result in mark deduction of 20%.
- Compress the directory into a single .zip file for submission.
- **Programming Practices:** Use well indented code, meaningful variable names, and clear comments. Understanding the problem statement is considered part of the evaluation.

Deadline: 8- MARCH-2026 at 12:00 AM

Question 1 : Simulation of Command echo user_str | wc c Using Unnamed Pipes

Write a program to simulate the shell pipeline command echo user_str | wc c, where the string "user_str" is provided by the user. The program should create two child processes. The first child process should read the user input string and send it to the parent process. The parent then forwards this string to the second child process, which counts the number of characters (similar to wc c) and sends the count back to the parent. Finally, the parent should display both the original string and its character count. All the communication between processes should be done using unnamed pipes.

Question 2: Rock Paper Scissors Game Using Named Pipes

Write two programs to implement a Rock Paper Scissors game played over 5 rounds between two players and a judge, using named pipes (FIFOs) for inter process communication. The programs must be run separately in different terminals(three), simulating independent processes.

Keypoints:

- Provides a clear illustration of named pipes (FIFOs) for inter-process communication between independent programs.
- Reinforces understanding of blocking behavior in open(), read(), and write().
- Encourages disciplined multi-terminal execution and clean resource management.
- Demonstrates synchronization using only operating system primitives without shared memory or signals.

Details

- The system must consist of three independent programs: one judge and two players, each executed in separate terminals. The judge must not fork or spawn player processes.
- Exactly two FIFOs must be used: /tmp/rps_p1 (Player 1 → Judge) and /tmp/rps_p2 (Player 2 → Judge). Communication is strictly unidirectional (players send choices to the judge only). The judge prints all round results and the final winner; no back-channel communication is required.
- The message format must be a single lower-case string terminated by a newline: "rock\n", "paper\n", or "scissors\n". Maximum message length is 16 bytes.
- The game consists of exactly five rounds. The judge reads one choice from each FIFO per round, displays both choices, determines the round winner using standard rules, updates scores, and prints the final result after five rounds.
- Player programs must validate user input locally and re-prompt on invalid entries. Invalid inputs must never be sent through the FIFO.
- Synchronization must rely solely on blocking behavior of FIFOs. The judge blocks while reading from each pipe, and players block if the FIFO is not yet opened. The required execution order is: start judge first, then Player 1, then Player 2.
- The judge must create FIFOs using mkfifo(), open them in read-only mode, and remove them using unlink() upon termination. Players must open their respective FIFO in write-only mode and close it after five rounds. Proper cleanup is mandatory.

```
Judge: Starting game...

Round 1:
Player 1 chooses: rock
Player 2 chooses: scissors
Result: Player 1 wins

Round 2:
Player 1 chooses: paper
Player 2 chooses: rock
Result: Player 1 wins

Round 3:
Player 1 chooses: scissors
Player 2 chooses: scissors
Result: Tie

Round 4:
Player 1 chooses: rock
Player 2 chooses: paper
Result: Player 2 wins

Round 5:
Player 1 chooses: scissors
Player 2 chooses: paper
Result: Player 1 wins

Overall Winner: Player 1
```

Question 3: Multi-Process Chess Simulation

Strengths

- Strong emphasis on process isolation and centralized state management through a controller process.
- Reinforces correct use of bidirectional pipes and explicit closure of unused file descriptors.
- Introduces logging, termination handling, and zombie process prevention using wait().
- Encourages structured communication protocols over byte-stream pipes.

System Overview

Develop a multi-process chess simulation where a **Controller process** manages the game state, while two **Player processes** (White and Black) make moves. The system enforces strict **process isolation**: players cannot access the global board state directly and must communicate **solely through pipes via the Controller**.

<u>Architecture & Requirements</u>

1. Controller Process

Initialization: Initialize an 8×8 chessboard using a simplified text-based representation.

Process Creation: Fork two child processes: Player White and Player Black.

Pipe Setup: Establish separate bidirectional communication channels:

Player White: Controller → Player, Player → Controller

Player Black: Controller → Player, Player → Controller

Game Loop:

1. Alternate turns between players.
2. Send the current board state to the active player over the pipe.
3. Block until the player sends a move.
4. Validate the move according to the simplified chess rules.
5. Update the board and log the move to game_log.txt.
6. Print the updated board, PIDs of the Controller and both players, and the specific move.

2. Player Processes

Execution: After forking, each player replaces its process image using exec():

Player White: execlp()

Player Black: execv()

Behavior:

1. Enter a loop:
   a) Read the board state from the Controller (blocking).
   b) If read() returns 0 (EOF), print "Game Over" and exit gracefully.
   c) Calculate a valid move (random or logic-based) and send it back to the Controller.
2. Close all unrelated pipe ends (e.g., Player 1 closes Player 2's pipes).

3. Simplified Chess Model & Move Rules

Allowed pieces: King, Queen, Rook, Bishop, Knight, Pawn

Removed rules: castling, en passant, pawn promotion, check/checkmate

Move Legality:

- Pawns move one square forward; initial two-square move allowed.
- Pieces cannot move through other pieces (except Knight).
- Kings cannot occupy the same square.
- Moves outside board limits or violating piece rules are illegal.
- Illegal moves must be rejected, and the player must resubmit a move.

4. Message Formats

- **Board State:** Fixed 64-byte array (8×8).

  Empty squares: .

  White pieces: uppercase letters

  Black pieces: lowercase letters

- **Move Message:** Fixed 5-character string, e.g., "E2E4\n"

  Columns: A–H (uppercase)

  Rows: 1–8

  Messages are newline-terminated.

5. Termination Conditions

A King is captured → game ends

20 total moves → draw

Illegal move → controller rejects and expects resubmission

Cleanup Protocol:

- Close write ends of both players' pipes (sends EOF).
- Call wait() for both child processes to prevent zombies.
- Print the final result.

6. Synchronization

Turn-taking is enforced via **blocking** read() **and** write() on pipes.

Controller is the sole authority for board state updates.

Proper pipe closure ensures EOF propagation and clean player termination.

Bonus Features (Optional, Advanced)

1. **Piece-Level Concurrency**

- Each Player forks a child for every live piece.
- Piece processes calculate possible moves and communicate them via pipes.
- Parent Player selects the best move and sends it to the Controller.

2. **Checkpointing & Resume (Crash Recovery)**

- Controller serializes board state and turn info after every move into checkpoint.bin.
- Controller can resume with --resume, restoring the board and instructing the correct player to continue.
- Players remain stateless; the Controller handles correct turn resumption.

Constraints

- IPC **must use pipes only**; signals and shared memory are forbidden.
- Proper closure of all unrelated pipe ends is **mandatory**.
- Code must handle pipe failures and unexpected player crashes robustly

Submission Requirements

- Source files: controller.c, player1.c, player2.c
- Makefile to compile all programs

Question 4: Intelligent Disk Cleanup Framework

Scenario

Your Linux workstation is critically low on disk space. You are required to design a robust, modular, and fault tolerant Bash based disk cleanup framework. The system must perform intelligent space analysis, safe cleanup operations, and storage optimization while handling large filesystems efficiently.

Functional Requirements

1. Large File Analysis

- **Scanning:** Recursively scan the current user's home directory to identify the **top 10 largest regular files**.
    - **Exclusions:** Symbolic links must be ignored. Mounted external filesystems must not be traversed.
- **Interaction:** For each identified file, display:
    - Absolute Path.
    - Human readable size.
    - Last Access and Modification timestamps.
- **Action Prompt:** The user must be prompted with the following options per file:
    - **Delete:** Remove the file.
    - **Compress:** Gzip the file in place.
    - **Move:** Relocate to a backup directory.
    - **Skip:** Take no action.
- **Automation:** The script must support a non interactive mode (via command line flags) and implement confirmation safeguards to prevent accidental deletion.

2. Duplicate Detection (Two Phase Mechanism)

- **Heuristic:** Implement a two phase comparison to minimize overhead:
    - **Phase 1:** Group files by size.
    - **Phase 2:** Verify duplicates within groups using **SHA 256** hashing.

- **Resolution:** Present identified duplicate groups to the user with options to:
    - **Delete:** Remove selected duplicates (ensuring one valid copy remains).
    - **Hard Link:** Replace duplicates with hard links to a single original (preserving inodes).
    - **Quarantine:** Move duplicates to a specific directory.
    - **Report:** Generate an analysis report without changes.
- **Safety constraints:**
    - Handle filenames with special characters correctly.
    - Strictly preserve file permissions and ownership during hard link operations.

3. System Sanitation Module

- **User Cache:** Analyze and optionally clean ~/.cache and browser temporary directories. Display estimated reclaimable space before deletion.
- **Privileged Operations (Root Only):**
    - **Log Cleaning:** Analyze /var/log. Identify logs older than a configurable threshold or exceeding a size limit. Offer to compress or delete them.
    - **Package Management:** Automatically detect the system package manager (apt, dnf, etc.) and execute the appropriate cleanup command (e.g., autoremove) upon user confirmation.
- **Privilege Check:** If the script is not executed as root, it must detect the lack of privileges and gracefully skip these operations with a warning.

Advanced Implementation Constraints

- **Structured Code:** All major features must be implemented as separate Bash functions.
- **Input Parsing:** Command line arguments must be handled using getopts.
- **Logging:** All operations must be logged with timestamps to cleanup.log.
- **Error Handling:**
    - Return proper exit codes for failure conditions.
    - Handle invalid paths and permission errors robustly.
    - Use trap handlers to clean up temporary files upon unexpected termination (e.g., SIGINT).
- **UX:** Provide colored terminal output for warnings, confirmations, and errors. Ensure efficiency when handling directories containing thousands of files.

Bonus Challenge: Stale File Heuristics

Implement an intelligent recommendation engine based on file access times:

- **Analysis:** Identify files not accessed in **>180 days** that exceed a configurable size threshold.

- **Heuristic:** Rank files using a scoring system that prioritizes older and larger files to estimate potential recovery.
- **Deliverables:**
    - Generate a deletion recommendation report.
    - Support a **Dry Run Mode** that simulates deletion without modifying the filesystem.
    - (Optional) Export the recommendation report to CSV format.

Thank you